

《软件新技术专题》大作业 · 终期答辩 · 2026-09-22

高校实验室资产与报修管理平台

面向高校二级学院实验室的「资产全生命周期 + 故障报修闭环」管理平台

第6组 · 软件工程 2023-2 · 授课教师 谭清萱

组员：王旻辉 · 巴力江 · 托力肯别克 · 廖煌 · 迪丽热巴 · 阿布都西克尔



背景 · 要解决的三个真实痛点

台账、工单、统计各自为政，其实是同一个缺口

三个场景的共同后果是没有履历 —— 查不出谁改的、催不到谁在修、算不出花了多少

痛点 01

台账靠 Excel 手工维护

资产调拨、报废没有留痕，学期末账实不符，查不出是谁改的

痛点 02

报修靠微信群喊人

谁报的、谁在修、修到第几步全靠记忆，超时无催办

痛点 03

没有统计数据

一学期修了多少次、哪台设备最常坏、维修费花了多少，完全说不清

一句话定位

面向高校二级学院实验室的「资产全生命周期 + 故障报修闭环」管理平台

自定义代码只住一个包，不污染若依脚手架

全部业务代码位于 com.ruoyi.project.laboratory，与若依原生模块物理隔离

展示层 Vue 3.5 · Element Plus 2.13	三个业务页 + 首页运维看板	views/laboratory/** views/index.vue
业务层 Spring Boot 2.5.15 · Security + JWT	四个业务领域各一套 Controller + Service	com.ruoyi.project.laboratory
持久化层 MyBatis · Mapper XML	自定义 Mapper 只读写自己的业务表	resources/mybatis/laboratory/*.xml
数据库层 MySQL 8 · education_system	5 张业务表 + 4 组字典，删除一律逻辑删除	lab_room · lab_asset · lab_repair · *_record

四个业务模块

- 实验室房间
- 资产台账
- 设备报修
- 运维工作台

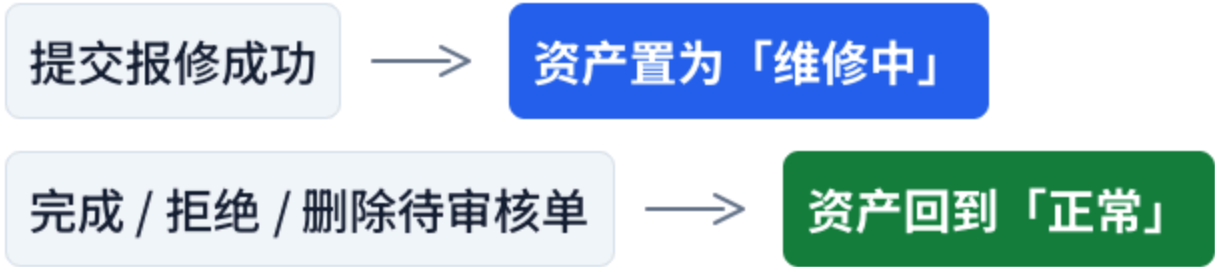
接口前缀 /laboratory/{room|asset|repair|dashboard}

资产状态由报修工单驱动，不允许手工改

流转规则只写在 validateStatusChange 一处，非法流转直接抛异常



资产状态联动 提交报修即锁定资产，闭环结束自动释放



主动不做两件事：预约模块、状态模式重构

范围不是越大越好 —— 每一条「不做」都有明确理由，答辩时主动说明

房间 / 设备预约与时段冲突

本期范围外，列为「后续迭代」

评分表五个维度，没有一个与功能数量相关

补一个模块要动表 + 后端 + 前端 + 菜单权限 + SQL —— 一整轮迭代的成本

用状态模式重构报修流转

主动判断：投入产出不划算

现有 validateStatusChange 只有 3 个分支的 if-else

引入状态模式属过度设计；讲「评估过、判断不划算」比堆模式更能得分

同样明确不做的 5 项

第三方支付与短信网关 · 小程序端 · 原生移动 App · 多校区 / 多租户隔离 · Selenium 端到端

AI 关掉了一个我们一直没发现的缺陷

一次「查缺补漏」巡检 —— 只给了一句提示词，AI 交回两个真实缺陷；而它自己算错的一处数字，被测试当场兜住

AI 帮我们发现的

首页看板对非超管角色返回 403

同一条权限散在四个技术层的五个位置，白名单不一致 —— 维修工程师、资产管理这类角色看不到本该可见的看板

处置 · 后端 LabRoleUtils 与前端常量对齐，SQL 末尾兜底补齐授权，登记为偏移 D-10

我们帮 AI 纠正的

非法流转断言：AI 写 11，实际 16

状态机共 5 个状态、有序对 25 组；AI 凭印象推定非法 11 组，正确推导是 $25 - 5 \text{ 组同状态幂等} - 4 \text{ 组合法} = 16 \text{ 组}$

处置 · 断言改为 16，测试立刻全绿 —— 若当初写成 `assert checked > 0`，这个错误会一直躺在代码里

这条权限要在 5 个位置说同一句话 —— 五处分布在四个技术层

① 接口注解

② 菜单权限

③ 角色授予

④ 前端指令

⑤ 行级过滤

把业务规则写成可断言的测试

49 个用例全部通过 —— 六条业务规则每一条都能指到一个具体的断言对象

端到端实测 · 不写脚本

73 条接口 0 失败

集成测试 16 个

H2 内存库 · 验落库结果

单元测试 33 个

Mockito 隔离 · 验调用契约

顶 · 为什么不写自动化脚本

现场跑真实环境，权限注解由 UI 巡检实测

中 · 为什么集成要 16 条

跨表事务、SQL 与表结构是否对得上 —— 单测发现不了

底 · 为什么单元最多

业务规则集中在 Service 层，分支与边界在这里穷举

如实说明

覆盖率百分比出不来 —— 本机离线仓库缺 JaCoCo 插件，改用「用例与业务规则」追溯矩阵作证据

从提交到上线，本地和 CI 跑同一套脚本

六段等价流水线，与 GitHub Actions 的 job 一一对应 —— 换环境不换命令

[1/6] 版本信息 PASS 3.5s	[2/6] 后端测试 PASS 111.6s	[3/6] 前端构建 PASS 49.5s	[4/6] 打包产物 PASS 49s	[5/6] 镜像构建 SKIP 0.1s	[6/6] 冒烟测试 SKIP 2.5s
本地开发 直连本机 MySQL 与 Redis 改端口或库名即可，不动代码	CI 流水线 跑 H2 内存库，零中间件依赖 与本地跑同一份测试命令	容器编排 四个服务同起，配置走环境变量 application.yml 一字未改			

本机限制 · 如实说明

本机未安装 Docker，第 5、6 段恒为 SKIPPED —— 脚本保证 SKIPPED 不伪装成 PASS

替代证据：94 条配置静态断言 + 23 条环境变量绑定实证，两组全绿

现场演示 · 三步闭环

三步走完一次报修闭环

从提交到评价，资产状态全程由工单驱动 —— 演示动作与截图一一对应

第一步 · 首页就是结论

① 登录 → 首页看板



4 个指标卡 + 4 张图；维修中资产 2 台

第二步 · 提报即联动资产

② 资产台账 → 报修



提交报修后，资产自动置「维修中」

第三步 · 闭环走完

③ 审核 → 完成 → 评价



状态机 0→1→2→3，完成后资产回到「正常」

演示账号 labadmin / 口令 admin123 —— 与 README、种子脚本三处口径已统一，现场登录不会翻车

同一条权限，只能有一处口径

角色白名单要在三处保持一致 —— 漏掉任意一处，就会出现「按钮看得到、接口却 403」

① 后端 · LabRoleUtils

两套白名单的唯一真源

canHandleRepair 与 canViewAll 两个方法；控制器与 Service 全部调它，不在别处复制判断

② 前端 · labPermission.js

同名常量照抄后端口径

两个常量与后端同名同义，文件注释里写明「必须与后端保持一致」，改一处要同步改另一处

③ SQL · permission_fix

末尾按角色清单兜底授权

菜单授权脚本里少了看板权限，非超管角色首页空白 —— 偏移 D-10 漏掉的正是这一处

可处理报修 · 4 / 8

admin

teacher

lab_manager

repair_engineer

可查看全部 · 6 / 8

admin

teacher

lab_manager

repair_engineer

asset_keeper

lab_viewer

蓝 = 与上行同一批角色；绿 = 只读观察角色。room_keeper / student_assistant 不在任一白名单 —— 只能看到自己提交的报修单

五张业务表，删除一律是逻辑删除

单据不消失、履历能回溯 —— 代价是不能给业务编号建唯一索引



删得了单据，删不掉行

列表查询一律带 `del_flag = '0'`；编号唯一性由 Service 校验承担，不建唯一索引

唯一索引会与「删掉后重建同编号」冲突 —— 实测返回 500，已用幂等迁移删掉这两个键

五条我们选择容忍的偏差

容忍不等于不知道 —— 每一条都写清了现象、影响与处置路径，答辩时主动说明

D-01	二维码是相对路径	手机相机扫不开，只能登录后站内跳转	配一个站点基址即可，属小改动
D-06	前后端状态机是两张手写表	跨语言无法真正共享，后端有矩阵测试兜底	改状态机时人工比对前端
D-08	签名密钥有可猜的默认值	默认值一旦上线就等于公开，能伪造登录凭证	用环境变量覆盖，写进部署文档
D-26	沙箱拦截 WMIC 会拆命令树	长流水线有概率被中途掐断，易误判成构建失败	脚本内置重试；CI 在 Linux 上无此问题
D-28	本机没装 Docker	容器未真跑，镜像与冒烟两段恒为 SKIPPED	改用静态断言与环境变量绑定实证

处置原则

容忍的标准是影响可控、且修复成本高于收益 —— 而像 D-10 那种用户可感知的缺陷，一律当场修掉

这套系统到底做了多少东西

每一个数字都能在仓库里用一条命令复算 —— 不给估算值

28

个 REST 接口，分在 4 个控制器

9

张数据表 —— 5 张业务 + 4 张字典

29

个生产类，全部收在一个自定义包内

49

个测试用例 —— 33 单元 + 16 集成

9

个 SQL 脚本，全部可重复执行

4

个业务模块 —— 房间 / 资产 / 报修 / 看板

口径说明

以上数字都能用一条命令复算 —— 测试数取自实跑日志，接口数取自注解计数

本次实跑记录 Tests run: 49, Failures: 0, Errors: 0, Skipped: 0 (提交 1668e4a

谁做了什么，AI 参与了哪一段

AI 的输出一律经人工复核后才进入交付，每一次改动都留有编号与留痕

组员分工 · 第 6 组 · 软件工程 2023-2

01 王旻辉 · 待认领

02 巴力江·托力肯别克 · 待认领

03 廖煌 · 待认领

04 迪丽热巴·阿布都西克尔 · 待认领

分工认领尚待确认（问题 Q-08）；确认后只替换本栏文字，页面结构不动

AI 使用声明 · 六个角色各领一块

产品经理 · 需求基线、任务卡与偏移台账

代码负责 · 前后端实现、CI/CD 与容器化

测试负责 · T1/T2，49 个用例全绿

文档负责 · 五份交付文档

UI 设计 · T6，界面统一与答辩截图

PPT 和设计 · D2，这份答辩稿

人工把关的环节

缺陷判定与流程纠偏、状态机与权限口径验收、容忍偏差的取舍决策

三条红线

查重率 ≤ 30%（含代码注释）

每份文档开头写明 AI 使用段落

答辩全员参与，教师随机点人